

Linear Regression: The features $x_1, x_2, \dots, x_D$ are known quantities.
The weights $w_1, w_2, \dots, w_D$ are the unknown quantities.
The terms linear or non-linear refer to the max powers of the weights w_i 's not x_i 's.

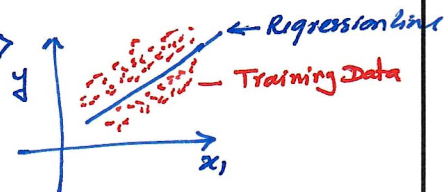
In the simplest case, y only depends on one variable i.e. $\vec{x} = \langle x_1 \rangle$

$$\hat{y} = h(\vec{x}) = w_0 + w_1 x_1$$

In general

$$\hat{y} = h(\vec{x}) = w_0 + w_1 x_1 + w_2 x_2 + \dots + w_D x_D, \text{ where } \vec{x} = \langle x_1, \dots, x_D \rangle$$

Weights $\vec{w} = \langle w_0, w_1, \dots, w_D \rangle$ are model params, learned from the data



Mathematical Representation

$\vec{x} = \langle 1, x_1, \dots, x_D \rangle \in \mathbb{R}^{D+1}$: Input; $D = \#$ Covariates/features/predictors

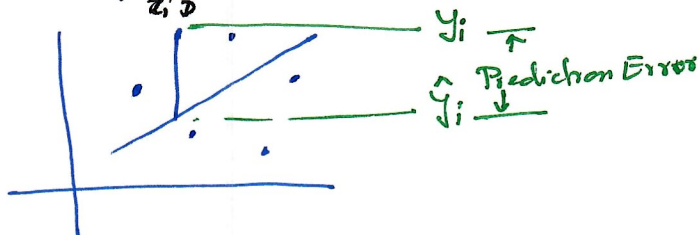
$\vec{w} = \langle w_0, w_1, \dots, w_D \rangle \in \mathbb{R}^{D+1}$: weights; model params

$D = \{(x_1, y_1), \dots, (x_D, y_D)\} \in \mathbb{R}^D$: Training Data, where $\vec{x}_i = i$ th input vector, $y_i =$ true i th response value.

$f: \vec{x} \rightarrow y, f(\vec{x}) = w_0 + \sum_{i=1}^D w_i x_i = \vec{w}^T \cdot \vec{x}$: Model

Recall that

$$\vec{x}_i = \begin{pmatrix} 1 \\ x_{i,1} \\ x_{i,2} \\ \vdots \\ x_{i,D} \end{pmatrix} \Rightarrow \vec{x}_i^T = [1, x_{i,1}, x_{i,2}, \dots, x_{i,D}] \quad \& \quad \vec{w}_i = \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_D \end{pmatrix} \Rightarrow \vec{w}_i^T = [w_0, w_1, \dots, w_D]$$



Residual Sum of Squares (RSS)

$$RSS(\vec{w}) = \sum_{n=1}^N \underbrace{(y_n - f(\vec{x}_n))}_{\text{Residue}}^2 = \sum_{n=1}^N \left[y_n - \left(w_0 + \sum_{d=1}^D w_d x_{n,d} \right) \right]^2$$

or

$$RSS(\vec{w}) = (\mathbf{y} - \mathbf{X}\vec{w})^T (\mathbf{y} - \mathbf{X}\vec{w})$$

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1D} \\ 1 & x_{21} & x_{22} & \dots & x_{2D} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{N1} & x_{N2} & \dots & x_{ND} \end{bmatrix} = \begin{bmatrix} \vec{x}_1^T \\ \vec{x}_2^T \\ \vdots \\ \vec{x}_N^T \end{bmatrix}$$

$$\mathbf{X}_{N \times (D+1)} \quad \vec{w}_{(D+1) \times 1} \Rightarrow \mathbf{X}\vec{w} = \begin{bmatrix} (\mathbf{y} - \mathbf{X}\vec{w})^T \\ \vdots \\ (\mathbf{y} - \mathbf{X}\vec{w})^T \end{bmatrix} \Rightarrow (\mathbf{y} - \mathbf{X}\vec{w})^T (\mathbf{y} - \mathbf{X}\vec{w})$$

Residue

Note: If $\vec{A} = \langle a_1, a_2 \rangle$, then
 $\|\vec{A}\| = \sqrt{a_1^2 + a_2^2} = \sqrt{\vec{A} \cdot \vec{A}} = \sqrt{\vec{A} \cdot \vec{A}^T}$

Closed Form Solution of Linear Regression

We want to find $\vec{w}$ such that $RSS(\vec{w})$ is minimized

For that we need to set $\frac{\partial}{\partial \vec{w}} (RSS(\vec{w})) = 0$

Notes From Matrix Algebra

$$1) (AB)^T = B^T A^T$$

$$2) \frac{\partial}{\partial \vec{b}} (\vec{b}^T A \vec{b}) = 2 A \vec{b}$$

$$3) \frac{\partial}{\partial \vec{a}} (\vec{b}^T \vec{a}) = \vec{b}$$

$$\begin{aligned} RSS(\vec{w}) &= (\vec{y} - X\vec{w})^T (\vec{y} - X\vec{w}) = (\vec{y}^T - \vec{w}^T X^T) (\vec{y} - X\vec{w}) = \vec{y}^T \vec{y} - \vec{w}^T X^T \vec{y} - \vec{y}^T X \vec{w} + \vec{w}^T X^T X \vec{w} \\ &= \vec{y}^T \vec{y} - \vec{y}^T X \vec{w} - \vec{w}^T X^T \vec{y} + \vec{w}^T X^T X \vec{w} \\ &= \vec{y}^T \vec{y} - \vec{y}^T X \vec{w} - \vec{w}^T X^T \vec{y} + \vec{w}^T X^T X \vec{w} \\ &= \vec{y}^T \vec{y} - \vec{y}^T X \vec{w} - \vec{w}^T X^T \vec{y} + \vec{w}^T X^T X \vec{w} \end{aligned}$$

$$\begin{aligned} &= \vec{y}^T \vec{y} - \vec{w}^T X^T \vec{y} - \vec{y}^T X \vec{w} + \vec{w}^T (X^T X) \vec{w} \\ &= \vec{y}^T \vec{y} - \vec{w}^T X^T \vec{y} - \vec{y}^T (X \vec{w}) + \vec{w}^T (X^T X) \vec{w} \\ &= \vec{y}^T \vec{y} - \vec{w}^T (X^T \vec{y}) - \vec{w}^T X^T \vec{y} + \vec{w}^T (X^T X) \vec{w} \\ &= \vec{y}^T \vec{y} - 2 \vec{w}^T (X^T \vec{y}) + \vec{w}^T (X^T X) \vec{w} = \vec{y}^T \vec{y} - 2 \underbrace{(X^T \vec{y})^T}_{\vec{b}^T \vec{a}} \vec{w} - \vec{w}^T (X^T X) \vec{w} \end{aligned}$$

$$\Rightarrow \frac{\partial}{\partial \vec{w}} RSS(\vec{w}) = 0 - 2 \underbrace{X^T \vec{y}}_{\text{From 3}} + 2 \underbrace{X^T X \vec{w}}_{\text{From 2}} = -2 \vec{b}^T \vec{a} + 2 \vec{b}^T \vec{a} \quad (4)$$

$$\Rightarrow \frac{\partial}{\partial \vec{w}} (RSS(\vec{w})) = -2 X^T \vec{y} + 2 X^T X \vec{w} = 0 \quad \text{For minimum}$$

$$\Rightarrow \vec{w} = (X^T X)^{-1} X^T \vec{y} \in \mathbb{R}^{D+1}$$

$X^T X$ is always a symmetric Matrix

Now let's ensure that the Hessian Matrix of $RSS(\vec{w})$ is positive semi-definite.
This will prove that $RSS(\vec{w})$ is a CONVEX Fn

$$\text{Hessian Matrix } H_{RSS(\vec{w})} = \frac{\partial^2}{\partial \vec{w}^2} RSS(\vec{w})$$

We need to prove that $\frac{\partial^2}{\partial \vec{w}^2} RSS(\vec{w})$ is Positive Semi-definite

Please see the next page

For a convex function, a local minimum is also a global minimum

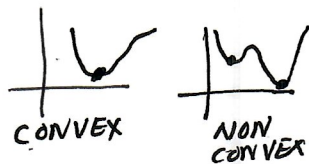
Theorem:

min $f(x)$ s.t. $x \in \Omega$ where f is a convex function, then any local minimum is also a global minimum

convex

Non-Convex

Theorem $\min f(x)$ such that $x \in \Omega$, where f is a convex fn & Ω is a convex set, then any local minimum is also a global minimum



A convex fn is one in which a line segment connecting any two points on its graph lies above or on the graph itself. (Has an open cup shape)

Convex Set



A set is convex set if any ^{line} vector connecting any two elements lies completely inside the set. (i.e. every point on the line is $\in$ the set)

Gradient Vector $\nabla f(\vec{x})$

for a $f: \mathbb{R}^D \rightarrow \mathbb{R}$, $\nabla f(\vec{x}) = \left[\frac{\partial f(\vec{x})}{\partial x_1}, \frac{\partial f(\vec{x})}{\partial x_2}, \dots, \frac{\partial f(\vec{x})}{\partial x_D} \right] \in \mathbb{R}^D$

Ex $f(\vec{x}) = f(x_1, x_2) = x_1^2 + x_2^2$

Then $\nabla f(\vec{x}) = [2x_1, 2x_2] \in \mathbb{R}^2$

Hessian Matrix H

for a fn $f: \mathbb{R}^D \rightarrow \mathbb{R}$, $H = \nabla \left((\nabla f(\vec{x}))^T \right) = \begin{bmatrix} \frac{\partial^2 f(\vec{x})}{\partial x_1^2} & \frac{\partial^2 f(\vec{x})}{\partial x_1 \partial x_2} & \dots & \frac{\partial^2 f(\vec{x})}{\partial x_1 \partial x_D} \\ \vdots & \frac{\partial^2 f(\vec{x})}{\partial x_2 \partial x_1} & \dots & \frac{\partial^2 f(\vec{x})}{\partial x_2 \partial x_D} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f(\vec{x})}{\partial x_D \partial x_1} & \frac{\partial^2 f(\vec{x})}{\partial x_D \partial x_2} & \dots & \frac{\partial^2 f(\vec{x})}{\partial x_D^2} \end{bmatrix} \in \mathbb{R}^{D \times D}$

Ex $f(\vec{x}) = x_1^2 + x_2^2$

$\nabla f(\vec{x}) = [2x_1, 2x_2]$

$\nabla \left((\nabla f(\vec{x}))^T \right) = \nabla \begin{bmatrix} 2x_1 \\ 2x_2 \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} (2x_1) & \frac{\partial}{\partial x_2} (2x_1) \\ \frac{\partial}{\partial x_1} (2x_2) & \frac{\partial}{\partial x_2} (2x_2) \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} \in \mathbb{R}^{2 \times 2}$

Positive Semi-Definite

A matrix A is positive semi-definite if $\vec{u}^T A \vec{u} \geq 0 \quad \forall \vec{u}$

Proof: $RSS(\vec{w})$ is a convex fn

We know $RSS(\vec{w}) = \vec{y}^T \vec{y} - 2 \vec{w}^T (X^T \vec{y}) + \vec{w}^T (X^T X) \vec{w}$

$\Rightarrow \frac{\partial RSS(\vec{w})}{\partial \vec{w}} = -2 (X^T \vec{y}) + 2 (X^T X) \vec{w}$

$\Rightarrow H_{RSS(\vec{w})} = \frac{\partial^2 RSS(\vec{w})}{\partial \vec{w}^2} = \frac{\partial}{\partial \vec{w}} \left(\frac{\partial RSS(\vec{w})}{\partial \vec{w}} \right)$

$= \frac{\partial}{\partial \vec{w}} \left(-2 (X^T \vec{y}) + 2 (X^T X) \vec{w} \right) = 2 (X^T X)$

For any $\vec{u}$, $\vec{u}^T H_{RSS(\vec{w})} \vec{u} = 2 \vec{u}^T (X^T X) \vec{u} = 2 (\vec{u}^T X^T) (X \vec{u}) = 2 (X \vec{u})^T (X \vec{u})$

$= 2 \|X \vec{u}\|_2^2 = 2 (\text{length of } X \vec{u})^2 \geq 0$

Recall

1) $(AB)^T = B^T A^T$

2) $\frac{\partial}{\partial \vec{B}} \frac{\partial}{\partial \vec{B}} (\vec{B}^T A \vec{B}) = 2 A \vec{B}$

3) $\frac{\partial}{\partial \vec{a}} (\vec{b}^T \vec{a}) = \vec{b}$

From 1

Recall
4) $\vec{a}^T \vec{a} = \|\vec{a}\|_2^2$

Since $\vec{u}^T H_{RSS(\vec{w})} \vec{u} \geq 0 \quad \forall \vec{u} \Rightarrow H_{RSS(\vec{w})}$ is Semi-Positive Semi-Definite matrix

$\Rightarrow RSS(\vec{w})$ is Convex

$\Rightarrow$ Any local minimum is also a global minimum.

$\therefore \boxed{\vec{w}^* = (X^T X)^{-1} X^T \vec{y} \in \mathbb{R}^D}$ is a Global minimum of the RSS error of the Linear Regression

So the solution $\vec{w}^* = (X^T X)^{-1} X^T \vec{y}$ is the Closed Form Solution

BUT its complexity is $O(D+1)^3$ using Gauss Jordan Elimination (since we need matrix multiplication & inverse)

$\therefore$ It is impractical for large D

NUMERICAL
Approximate Solution using Iterative Gradient Descent (derivative or gradient is tangent line/plane/xy/plane etc..)

Points to remember Gradient $\nabla f(\vec{x}) = [\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_D}]$ always points in the direction of MAXIMUM ASCENT. (This is a vector)

The rate of change of $f(\vec{x})$ in the direction of a unit vector $\hat{u}$ is $= \nabla f(\vec{x}) \cdot \hat{u} \dots$ (This is a number.)

Since $-\nabla f(\vec{x})$ (negative of Gradient Vector) points to the direction of the Steepest Descent

- We try to find the function's minimum via local search
- Std. Optimization algo in many ML application
 - Linear Regression, Logistic Regression
 - Scales well for big D

Please see Next page

Aside for $z = f(x)$ $\frac{\partial z}{\partial x} \big|_{x=x_0}$ is the value of how fast z changes with x

for $z = f(x, y)$, $\vec{\nabla} z = \langle \frac{\partial z}{\partial x}, \frac{\partial z}{\partial y} \rangle$ is how fast z changes with x & y

$\nabla z \cdot \hat{u} =$ A number that gives how fast z changes in the direction of $\hat{u}$

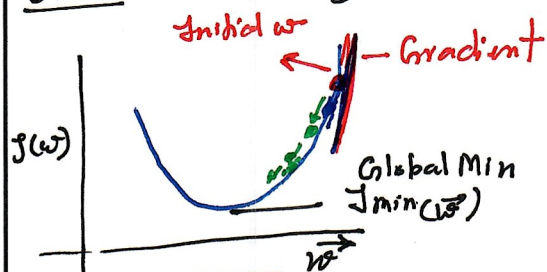
Iterative Gradient Descent Details

Input: Function $J(\vec{w}) : J: \mathbb{R}^D \rightarrow \mathbb{R}$

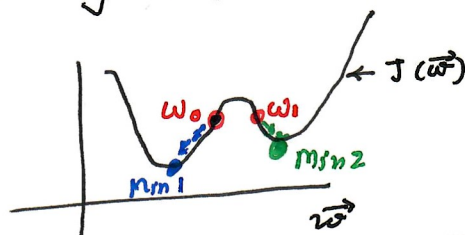
Output: Local Minimum $\vec{w}^*$

Goal: Minimize $J(\vec{w})$ with Greedy Local Search

Greedy $\rightarrow$ Makes
Locally optimal
choices



1-D ex. $\vec{w} \in \mathbb{R}^1$
with Convex $F'n$



1-D ex with non-convex $F'n$

The solution depends on whether we start at initial weight w_0 or w_1 & we will arrive either $\min_1$ or $\min_2$

Recall that 1) $J'(\vec{w}) = \frac{\partial J(\vec{w})}{\partial \vec{w}} = \frac{J(\vec{w} + \epsilon) - J(\vec{w})}{\epsilon}$, $\epsilon \rightarrow 0$ (Defn of derivative)

2) $J'(\vec{w})|_{\vec{w}^*} = 0$ ($J'(\vec{w}) = 0$ for min or max)

Algorithm

While $J'(\vec{w}) \neq 0$:

If $J'(\vec{w}) > 0$, and $J(\vec{w})$ is increasing, then $\uparrow$ decreases $\vec{w}$ a bit

If $J'(\vec{w}) < 0$ i.e. $J(\vec{w})$ is decreasing, then move a bit to right (increase a bit)

i.e. we ALWAYS move towards the direction of $-J'(\vec{w})$ i.e. opposite to $J'(\vec{w})$

more concrete Pseudo Algorithm

1-Dimensional

Initialize $w, \epsilon, \alpha(\cdot)$ Learning Rate

$K := 0$

While $\left| \frac{dJ(w)}{dw} \right| > \epsilon$:

$K = K + 1$

$w = w - \alpha(K) \frac{dJ(w)}{dw}$

* Change in w is only in ONE direction

D-Dimensional

Initialize $\vec{w}, \epsilon, \alpha(\cdot)$ Learning Rate

$K := 0$

While $\|\nabla J(\vec{w})\|_2 > \epsilon$:

$K = K + 1$

$\vec{w} = \vec{w} - \alpha(K) \cdot \nabla J(\vec{w})$

* change in $\vec{w}$ is in MULTIPLE directions.

Aside when we say $J(w)$ is increasing, it actually means that it increases with increasing w

If the LR α is small, we converge slowly
 If the LR α is too big, we can overshoot

Note LR α is a hyperparam
 ϵ is also a hyperparam
 both are set PRIOR to training
 $\vec{w}$ is model param, learned from data

Gradient Descent For Linear Regression

Recall that we want to minimize our loss $J(\vec{w}) = \text{RSS}(\vec{w})$ — RSS = Residual Sum of Squares

$$\text{So } J(\vec{w}) = \vec{y}^T \vec{y} - 2\vec{w}^T (X^T \vec{y}) + \vec{w}^T (X^T X) \vec{w}$$

$$\text{and } \nabla J(\vec{w}) = \frac{\partial \text{RSS}(\vec{w})}{\partial \vec{w}} = -2X^T \vec{y} + 2X^T X \vec{w}$$

$$= 2(X^T X \vec{w} - X^T \vec{y})$$

Remember that

$$1) (AB)^T = B^T A^T$$

$$2) \frac{\partial}{\partial \vec{B}} (\vec{B}^T A \vec{B}) = 2A\vec{B}$$

$$3) \frac{\partial}{\partial \vec{a}} (\vec{B}^T \vec{a}) = \vec{B}$$

$$4) \vec{a}^T \cdot \vec{a} = \|\vec{a}\|_2^2$$

Batch Gradient Descent

Initialize $\vec{w}, \epsilon, K=0$

While $\|\nabla \text{RSS}(\vec{w})\| > \epsilon$

$K := K+1$

$$\vec{w} = \vec{w} - \alpha(K) \cdot (X^T X \vec{w} - X^T \vec{y})$$

depend on

- Feature Matrix of ALL Training data

- $\vec{y}$ for all training data

- EXPENSIVE — ①

Stochastic Gradient Descent

Initialize $\vec{w}, \epsilon, K=0$

Loop until convergence

$K := K+1$

Randomly choose one sample $(\vec{x}_i, y_i)$ from training data

↳ that's why STOCHASTIC

Compute its contribution to the gradient

$$\vec{g}_i = (X_i^T \vec{w} - y_i) \vec{x}_i$$

$$\vec{w} = \vec{w} - \alpha(K) \cdot \vec{g}_i$$

Computationally inexpensive BUT
 (x_i, y_i) might be an outlier or outlier OR
 $\vec{g}_i$ may not be significant enough
 slow or impossible — ② convergence

Mini Batch Gradient Descent

To address ① & ②, we find a middle path, use m random samples

Initialize $\vec{w}, \epsilon, \alpha(\cdot), K=0$

Loop until convergence

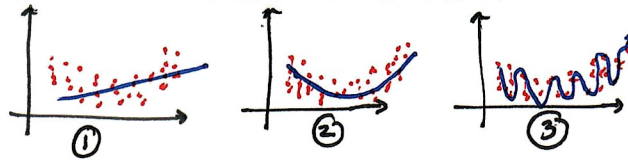
Randomly choose a subset of samples $S = \{(x_i, y_i), \dots, (x_{i+m}, y_{i+m})\}$

Form Mini-Batch Matrix $X_S = \begin{bmatrix} x_1^T \\ \vdots \\ x_{i+m}^T \end{bmatrix}$

$$\vec{w} = \vec{w} - \alpha(K) [X_S^T X_S \vec{w} - X_S^T \vec{y}]$$

Good compromise

m is usually between 50-250

Non-Linear Regression

Compare the three models.

- ① Linear model may not be able to capture the variance in the data
- ② might reduce $RSS(\vec{w})$ the most but is likely to OVERFIT the data i.e. it may be too sensitive to noise and capture too much detail
- ③ Might be the best fit - Captures the data variance without overfitting.

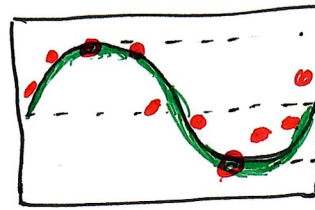
Non-Linear Basis Function

• When the data does NOT fit a line

• Non-Linear Basis Function ϕ ;

$$\phi(\vec{x}) : \vec{x} \in \mathbb{R}^D \rightarrow \vec{z} \in \mathbb{R}^M$$

ϕ maps D -dim input space $\vec{x}$ to M -dimensional input space $\vec{z}$, where $M > D$



• $\rightarrow$ Training Data
• $\rightarrow$ True Curve
Assume that we generate random data from a single

We now apply linear regression model to the new features $\vec{z}$

$$y_i = \vec{w}^T \vec{z}_i = \vec{w}^T \phi(\vec{x}_i), \quad \vec{w} \in \mathbb{R}^M \text{ now}$$

$$RSS(\vec{w}) = \sum_{i=1}^N (y_i - \vec{w}^T \phi(\vec{x}_i))^2, \quad \vec{w} \in \mathbb{R}^M \text{ now}$$

Example Let $\vec{x} = [x_1, x_2]^T \in \mathbb{R}^2$ & $\phi(\vec{x}) = [x_1, x_1^2, x_1^3 + x_2]^T \in \mathbb{R}^3$

i.e. $\phi(\vec{x}) \in \mathbb{R}^M$ where $M > D$ & also $\phi(\vec{x})$ has higher order terms of x_1 & x_2

Now $RSS(\vec{w}) = \sum_{i=1}^N (y_i - \vec{w}^T \phi(x_i))^2 = (\vec{y} - \Phi \vec{w})^T (\vec{y} - \Phi \vec{w})$ This is the same form as we saw before, except that $\vec{w} \in \mathbb{R}^M$ now

Non-Linear Design Matrix $\Phi = \begin{bmatrix} \phi(x_1)^T \\ \vdots \\ \phi(x_N)^T \end{bmatrix} \in \mathbb{R}^{N \times M}$

Least Mean Square (LMS) Solution

$$\vec{w}^{LMS} = (\Phi^T \Phi)^{-1} \Phi^T \vec{y}$$

This transformation would ensure that $\vec{w}^{LMS}$ is back to D -Dimensional

Contd.: Non-Linear Basis F'n

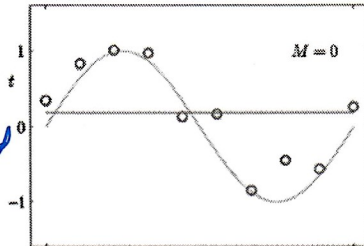
Ex - Samples from a sine function Polynomial Basis F'n $\phi(x) = [1, x, x^2, \dots, x^M]^T$

Example: Samples from a sine function

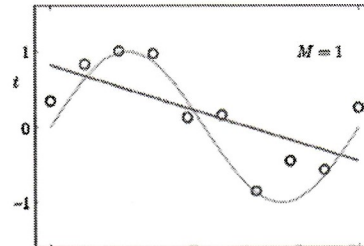
Polynomial basis function $\phi(x) = [1 \ x \ \dots \ x^M]^T$

Only 1 weight
(Bias term) to
learn
y DOES NOT depend
on x, when $M=0$

$M=0$ underfitting



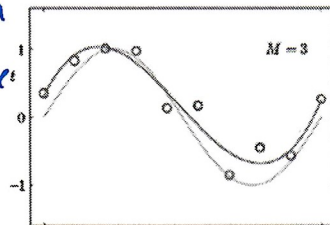
$M=1$ underfitting



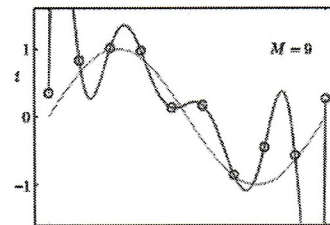
2 weights to learn

4 weights to learn
But compromise:

$M=3$



$M=9$ overfitting



10 weights to learn

When a model overfits, the weights start becoming VERY large.

$$y_i = W^T z_i = W^T \phi(x_i), \quad z_i = \phi(x_i) \in \mathbb{R}^M$$

Ex (Sine Curve data above)

	$M=0$	$M=1$
w_0	0.19	0.82
w_1		-1.27
w_2		
w_3		
w_4		
$\vdots$		
w_{10}		

$m=3$

0.31
7.49
-25.33
17.33

$m=9$

0.38
232.37
-5321.83
48518.3

125201.43

Too large

Please see Next Page

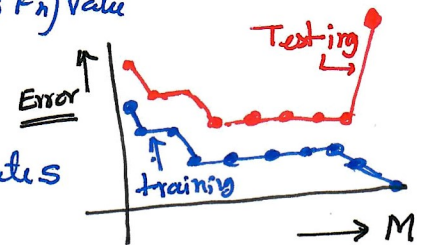
Overfitting

$$\phi(\vec{x}) : \vec{x} \in \mathbb{R}^D \rightarrow \vec{z} \in \mathbb{R}^m, m > 0$$

- Makes the model more flexible
- Makes the model more complex
- May lead to overfitting & Poor generalization

Detecting Overfitting

- Plot model complexity (polynomial Order) V/s Objective (i.e. Loss F_n) Value
- As Complexity increases
 - Performance on training improves
 - Performance on testing improves then deteriorates

Addressing overfitting

$$y = w_0 + w_1 x + w_2 x^2 + \dots + w_m x^m$$

- More data: The weight $w_0, \dots, w_m$ become more reasonable
- The data variance is captured without overfitting
- Regularization: Increased complexity results in too large values of weights w_i
 - Penalize model complexity by adding regularization
 - Add to loss function terms that would result in place constraints on the large values of weights.
 - Since we do not know which w_i to penalize, penalize them all

Regularization: Modify the Loss F_n L2 Norm Regularization

$$J(\vec{w}) = \text{RSS}(\vec{w}) + \lambda \sum_{d=1}^D w_d^2$$

$$J(\vec{w}) = \text{RSS}(\vec{w}) + \lambda \|\vec{w}\|_2^2$$

$$J(\vec{w}) = \text{RSS}(\vec{w}) + \frac{\lambda}{N} \|\vec{w}\|_2^2$$

Since we need less regularization as N increases

L1 Norm Regularization

$$J(\vec{w}) = \text{RSS}(\vec{w}) + \lambda \|\vec{w}\|_1$$

$\lambda \rightarrow$ Hyperparameter.

L2 Norm Regularization is used in
- Linear Regression, Logistic Regression
& neural networks.

Notation

Linear

$$J(\vec{w}) = \text{RSS}(\vec{w}) + \lambda \|\vec{w}\|_2^2$$

$$= (\mathbf{y} - \mathbf{X}\vec{w})^T (\mathbf{y} - \mathbf{X}\vec{w}) + \lambda \|\vec{w}\|_2^2$$

Closed

$$\vec{w}^* = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}_{D \times D})^{-1} \mathbf{X}^T \mathbf{y}$$

Reduces to OLS (Ordinary Least Squares)
when $\lambda = 0$

L2 Norm Regularization

Non-Linear

$$J(\vec{w}) = (\mathbf{y} - \phi(\vec{w}))^T (\mathbf{y} - \phi(\vec{w})) + \lambda \|\vec{w}\|_2^2$$

Form Soln

$$\vec{w}^* = (\phi^T \phi + \lambda \mathbf{I}_{D \times D})^{-1} \phi^T \mathbf{y}$$

Reduces to OLS (Ordinary Least Squares)
when $\lambda = 0$

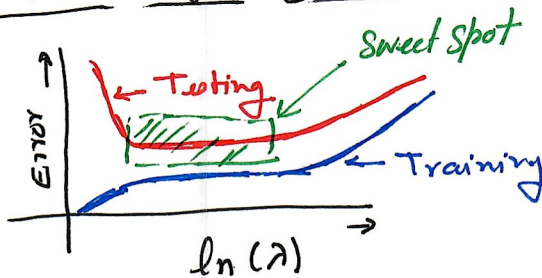
Another method to address overfitting is Feature Selection

We saw that non-linear regression is prone to overfitting.

Similarly Linear Regression may need regularization as well because

- Collinearity of multicollinearity
 - Near linear correlation among features
 - Same variable expressed in two different units.
 - $x_1 + x_1 + x_3 = 100\%$ kind of thing.
- multicollinearity may cause Matrices to be non-invertible.
 - Inaccurate estimation of regression coefficients
- Regularization would bring some coefficients closer to 0 (zero).
 $\rightarrow$ ~~Am~~ Involved in multicollinearity

Effect of Regularization



More Regularization (bigger λ) means less complex model.
 $\rightarrow$ Hyper param

$\Rightarrow$ Too much regularization leads to too simplistic model causing the error to go up.